Emulation of Compute Express Link (CXL) on QEMU via Ubuntu 24.04.4 LTS

Phase 1: Host System Preparation and Dependency Resolution

The foundation of the emulation environment is that the host must be running Ubuntu 24.04.4 LTS (Noble Numbat).

Workspace Initialization & KVM Setup

The initial step requires the creation of an isolated directory structure within the root filesystem, and ensuring the user has KVM hypervisor privileges.

Execute the following commands in the terminal. The path to execute these commands is the root directory /.

```
cd /
sudo mkdir -p /opt/cxl_workspace/qemu_build
sudo mkdir -p /opt/cxl_workspace/kernel_build
sudo mkdir -p /opt/cxl_workspace/images
sudo mkdir -p /opt/cxl_workspace/tools
sudo mkdir -p /opt/cxl_workspace/qemu_install
sudo chown -R $USER:$USER /opt/cxl_workspace

# Grant the user KVM privileges for hardware acceleration
sudo usermod -aG kvm $USER

# Apply the new group membership to the current shell environment
newgrp kvm

# Verify KVM access (should output the 'kvm' group)
groups | grep kvm
```

System Update and Core Dependency Installation

Ubuntu 24.04.4 LTS relies on the Advanced Package Tool (APT). Before installing specific dependencies, the system's package index must be updated.

The path to execute these commands is /opt/cxl_workspace.

```
cd /opt/cxl_workspace
sudo apt-get update
sudo apt-get dist-upgrade -y
```

The installation matrix for QEMU, the Linux kernel, and the user-space management utilities spans dozens of C/C++ libraries.

The path to execute this command is /opt/cxl_workspace.

```
cd /opt/cxl_workspace
sudo apt-get install --no-install-recommends -y \
  build-essential git ccache flex bison bc pkg-config \
  automake autoconf libtool ninja-build meson cmake \
  python3 python3-venv python3-pip python3-dev \
  python3-sphinx python3-sphinx-rtd-theme \
  ca-certificates wget curl \
  libglib2.0-dev libpixman-1-dev zlib1g-dev libgcrypt20-dev \
  libfdt-dev libffi-dev libslirp-dev liburing-dev libnfs-dev \
  libcurl4-gnutls-dev libzstd-dev libgudev-1.0-dev libaio-dev \
  libpmem-dev libpmem2-dev libssh-dev dbus-daemon dwarves perl \
  bridge-utils libncurses-dev libssl-dev libelf-dev libudev-dev \
  libpci-dev llvm clang asciidoc asciidoctor ruby-asciidoctor \
  xmlto libkmod-dev libsystemd-dev uuid-dev libjson-c-dev \
  libkeyutils-dev libiniparser-dev libtraceevent-dev libtracefs-dev \
  libnl-3-dev libnl-route-3-dev libibverbs-dev librdmacm-dev \
  libusb-1.0-0-dev libepoxy-dev libdrm-dev libgbm-dev libegl1-mesa-dev \
  ovmf qemu-utils libguestfs-tools \
  socat numactl iproute2 netcat-openbsd \
  sparse cscope exuberant-ctags \
  libvirglrenderer-dev libsdl2-dev libgtk-3-dev \
  libvte-2.91-dev libpulse-dev libjack-dev \
  libspice-protocol-dev libspice-server-dev \
  xfslibs-dev libbpf-dev

sudo update-guestfs-appliance

# Pull in any remaining hidden QEMU build dependencies native to Ubuntu
sudo sed -i 's/^Types: deb$/Types: deb deb-src/' /etc/apt/sources.list.d/ubuntu.sources
sudo apt-get update
sudo apt-get build-dep qemu -y
```

Phase 2: Building QEMU with Advanced CXL 3.1 and DCD Extensions

The definitive source for experimental CXL 3.1 features is heavily developed and staged in the fork maintained by Jonathan Cameron.

Fetching the Emulator Source Code

The path to execute these commands is /opt/cxl_workspace/qemu_build.

```
cd /opt/cxl_workspace/qemu_build
git clone https://gitlab.com/jic23/qemu.git
cd qemu

# Checkout the latest CXL integration branch dynamically.
git checkout -b cxl-stable origin/cxl-2025-03-20

# Initialize all submodules
git submodule update --init --recursive
```

Configuring and Compiling the QEMU Binary

The configuration script must be explicitly directed to target the x86_64 architecture for the system emulator.

The path to execute these commands is /opt/cxl_workspace/qemu_build/qemu.

```
cd /opt/cxl_workspace/qemu_build/qemu
./configure \
  --target-list=x86_64-softmmu \
  --enable-debug \
  --enable-slirp \
  --enable-kvm \
  --enable-vhost-net \
  --enable-libpmem \
  --enable-virtfs \
  --enable-linux-aio \
  --enable-bpf \
  --prefix=/opt/cxl_workspace/qemu_install
```

The path to execute these commands is /opt/cxl_workspace/qemu_build/qemu.

```
# From your qemu source directory
sed -i 's/unsigned int kind/ReplayClockKind kind/g' stubs/replay-tools.c

cd /opt/cxl_workspace/qemu_build/qemu
make -j$(nproc)
make install

# Verify the CXL-capable binary exists and is the correct version
```

```
/opt/cxl_workspace/qemu_install/bin/qemu-system-x86_64 --version

# Verify the binary exposes the required CXL devices
/opt/cxl_workspace/qemu_install/bin/qemu-system-x86_64 -device help 2>&1 | grep -i cxl
```

Phase 3: Building a Custom CXL-Aware Linux Kernel

For the guest operating system to interpret and interact with the complex CXL hardware hierarchy presented by the QEMU emulator, the Linux kernel must be compiled with CXL specific subsystem configurations. Pin the target kernel to **version 6.18 LTS**.

Fetching the Linux Kernel Source

The path to execute these commands is /opt/cxl_workspace/kernel_build.

```
cd /opt/cxl_workspace/kernel_build
git clone --depth=1 --branch v6.18 https://github.com/torvalds/linux.git
cd linux
```

Kernel Configuration for Advanced CXL Emulation

Start from a KVM-guest-optimized defconfig, and subsequently utilize the scripts/config utility to programmatically inject the mandatory CXL subsystems.

The path to execute these commands is /opt/cxl_workspace/kernel_build/linux.

```
cd /opt/cxl_workspace/kernel_build/linux
make defconfig
make kvm_guest.config
```

The path to execute these commands is /opt/cxl_workspace/kernel_build/linux.

```
cd /opt/cxl_workspace/kernel_build/linux

./scripts/config --enable CONFIG_EXPERT
./scripts/config --enable CONFIG_CXL_BUS
./scripts/config --enable CONFIG_CXL_PCI
./scripts/config --enable CONFIG_CXL_ACPI
./scripts/config --enable CONFIG_CXL_PMEM
./scripts/config --enable CONFIG_CXL_MEM
./scripts/config --enable CONFIG_CXL_PORT
./scripts/config --enable CONFIG_CXL_REGION
./scripts/config --enable CONFIG_CXL_MEM_RAW_COMMANDS
./scripts/config --enable CONFIG_CXL_SUSPEND
```

```
./scripts/config --enable CONFIG_ZONE_DEVICE
./scripts/config --enable CONFIG_DEV_DAX
./scripts/config --enable CONFIG_DEV_DAX_CXL
./scripts/config --enable CONFIG_DEV_DAX_PMEM
./scripts/config --enable CONFIG_FS_DAX
./scripts/config --enable CONFIG_LIBNVDIMM
./scripts/config --enable CONFIG_BLK_DEV_PMEM
./scripts/config --enable CONFIG_MEMORY_HOTPLUG
./scripts/config --enable CONFIG_MEMORY_HOTPLUG_DEFAULT_ONLINE
./scripts/config --enable CONFIG_MEMORY_HOTREMOVE
./scripts/config --enable CONFIG_NUMA
./scripts/config --enable CONFIG_ACPI_NUMA
./scripts/config --enable CONFIG_EXT4_FS
./scripts/config --enable CONFIG_DEBUG_FS
./scripts/config --enable CONFIG_CXL_REGION_INVALIDATION_TEST
```

Kernel Compilation

The path to execute these commands is /opt/cxl_workspace/kernel_build/linux.

```
cd /opt/cxl_workspace/kernel_build/linux
make olddefconfig

# Ensure the destination directory exists before attempting to install modules
mkdir -p /opt/cxl_workspace/images/modules

make -j$(nproc) bzImage modules
make modules_install INSTALL_MOD_PATH=/opt/cxl_workspace/images/modules
```

The resulting compressed kernel executable is located at arch/x86/boot/bzImage.

Phase 4: UEFI Firmware Integration via OVMF

The firmware initializing the virtual hardware must generate the CXL Early Discovery Table (CEDT). The default QEMU firmware, SeaBIOS, cannot do this. **OVMF is a strict non-negotiable requirement.**

Obtaining and Preparing the OVMF Binaries

Because the virtual machine writes UEFI boot variables to the variable store (OVMF_VARS.fd), we must copy pristine instances of these templates into the isolated workspace.

The path to execute these commands is /opt/cxl_workspace/images.

```
cd /opt/cxl_workspace/images

# Verify paths using a proper single-line fallback
ls /usr/share/OVMF/OVMF_*.fd 2>/dev/null || find /usr/share -name 'OVMF_CODE.fd' 2>/dev/null

# EXTREMELY IMPORTANT: Ensure the space exists before the dot '.' to copy to current directory
cp /usr/share/OVMF/OVMF_CODE*.fd .
cp /usr/share/OVMF/OVMF_VARS*.fd .

# Standardize names for the QEMU execution script using safe single-line fallback logic
mv OVMF_CODE_4M.fd OVMF_CODE.fd 2>/dev/null
mv OVMF_VARS_4M.fd OVMF_VARS.fd 2>/dev/null

# Verify they are regular files, not symlinks
file ./OVMF_CODE.fd
```

Phase 5: Guest OS Root Filesystem Construction

Using a minimal Ubuntu Noble Numbat cloud image.

Downloading and Expanding the Virtual Disk

The path to execute these commands is /opt/cxl_workspace/images.

```
cd /opt/cxl_workspace/images
wget https://cloud-images.ubuntu.com/noble/current/noble-server-cloudimg-amd64.img

qemu-img convert -O qcow2 noble-server-cloudimg-amd64.img cxl-guest.qcow2
qemu-img resize -f qcow2 cxl-guest.qcow2 20G
```

Hardening the Guest Image for Direct Boot and CXL Development

Cloud images rely on cloud-init for metadata ingestion, which we bypass. To ensure the guest boots flawlessly and possesses network connectivity for installing our tools, we must inject a password, generate a network configuration, enforce SSH rules, and synchronize our custom kernel modules.

We explicitly use LIBGUESTFS_BACKEND=direct to circumvent any nested-KVM permission panics when virt-customize prepares its supermin appliance.

The path to execute these commands is /opt/cxl_workspace/images.

```
cd /opt/cxl_workspace/images
```

```
# 1. Create a static Netplan configuration file
cat << 'EOF' > 01-dhcp.yaml
network:
  version: 2
  renderer: networkd
  ethernets:
    all_eth:
      match:
        name: "e*"
      dhcp4: true
EOF

# Enable highly compatible direct mode for libguestfs
export LIBGUESTFS_BACKEND=direct

# 2. Inject root password, bypass cloud-init, and setup console
sudo virt-customize -a cxl-guest.qcow2 --root-password password:cxladmin
sudo virt-customize -a cxl-guest.qcow2 --run-command 'touch /etc/cloud/cloud-init.disabled'
sudo irt-customize -a cxl-guest.qcow2 --run-command 'systemctl enable serial-getty@ttyS0.service'

# 3. Re-enable root SSH password login (disabled by default on cloud images)
sudo virt-customize -a cxl-guest.qcow2 --run-command 'sed -i "s/^#*PermitRootLogin.*/PermitRootLogin yes/" /etc/ssh/sshd_config'
sudo virt-customize -a cxl-guest.qcow2 --run-command 'sed -i "s/^#*PasswordAuthentication.*/PasswordAuthentication yes/" /etc/ssh/sshd_config'

# 4. Inject the Netplan config to guarantee network on boot
virt-customize -a cxl-guest.qcow2 --copy-in 01-dhcp.yaml:/etc/netplan/

# 5. Synchronize the custom 6.18 kernel modules into the guest's /lib directory
virt-customize -a cxl-guest.qcow2 --copy-in /opt/cxl_workspace/images/modules/lib/modules:/lib/
```

Phase 6: Executing QEMU CXL Emulation Topologies

We implement -cpu host,migratable=off to ensure migration boundaries do not mask hardware capability features. Crucially, because we are booting our custom CXL kernel *directly* without an initramfs (which normally extracts block UUIDs asynchronously), we **must** append root=LABEL=cloudimg-rootfs rootwait. rootwait instructs the kernel to patiently wait for the VirtIO block device to initialize before mounting, completely eliminating VFS kernel panics.

Before executing QEMU, create the physical backing files and flush old QMP sockets. Note that we do not dd the volatile memory backend, as QEMU maps that directly to system RAM.

The path to execute these commands is /opt/cxl_workspace/images.

```
cd /opt/cxl_workspace/images
dd if=/dev/zero of=cxl-mem0.raw bs=1M count=1024
dd if=/dev/zero of=cxl-lsa0.raw bs=1M count=256
dd if=/dev/zero of=cxl-vmem0.raw bs=1M count=512

# Prevent unintended modifications on multi-user systems
chmod 660 /opt/cxl_workspace/images/cxl-*.raw

[ -e /tmp/qmp-sock ] && rm -f /tmp/qmp-sock
```

Some other commands:

1. Fix the networkd-wait-online hang permanently(Execute this inside VM):
   ```
   systemctl disable systemd-networkd-wait-online.service
   systemctl mask systemd-networkd-wait-online.service
   ```

2. ```
   cd /opt/cxl_workspace/images
   sudo virt-customize -a cxl-guest.qcow2 --run-command 'apt-get install -y ndctl'
   ```

Emulation Scenario: Standard CXL Type 3 Memory Expansion

This topography models a single host bridge, a single root port, and one directly attached CXL Type 3 Persistent Memory endpoint.

The path to execute this command is /opt/cxl_workspace/images.

```
cd /opt/cxl_workspace/images

/opt/cxl_workspace/qemu_install/bin/qemu-system-x86_64 \
  -machine q35,cxl=on,accel=kvm \
  -cpu host,migratable=off \
  -smp 4 \
  -m 8G,maxmem=16G,slots=4 \
  -drive if=pflash,format=raw,readonly=on,file=/opt/cxl_workspace/images/OVMF_CODE.fd \
  -drive if=pflash,format=raw,file=/opt/cxl_workspace/images/OVMF_VARS.fd \
  -kernel /opt/cxl_workspace/images/bzImage-6.18-cxl \
  -append "root=/dev/vda1 rootwait rootdelay=5 console=ttyS0,115200 earlyprintk=ttyS0 rw" \
  -drive file=/opt/cxl_workspace/images/cxl-guest.qcow2,format=qcow2,if=none,id=hd0 \
  -device virtio-blk-pci,drive=hd0,bus=pcie.0 \
```
```

```
  -netdev user,id=net0,hostfwd=tcp::2222-:22 \
  -device virtio-net-pci,netdev=net0,bus=pcie.0 \
  -nographic \
  -d guest_errors \
  -object memory-backend-file,id=cxl-mem0,share=on,mem-
path=/opt/cxl_workspace/images/cxl-mem0.raw,size=1G \
  -object memory-backend-file,id=cxl-lsa0,share=on,mem-
path=/opt/cxl_workspace/images/cxl-lsa0.raw,size=256M \
  -device pxb-cxl,bus_nr=12,bus=pcie.0,id=cxl.1 \
  -device cxl-rp,port=0,bus=cxl.1,id=root_port13,chassis=0,slot=2 \
  -device cxl-type3,bus=root_port13,persistent-memdev=cxl-mem0,lsa=cxl-lsa0,id=cxl-
pmem0,sn=0x1 \
  -M cxl-fmw.0.targets.0=cxl.1,cxl-fmw.0.size=4G,cxl-fmw.0.interleave-granularity=4k
```